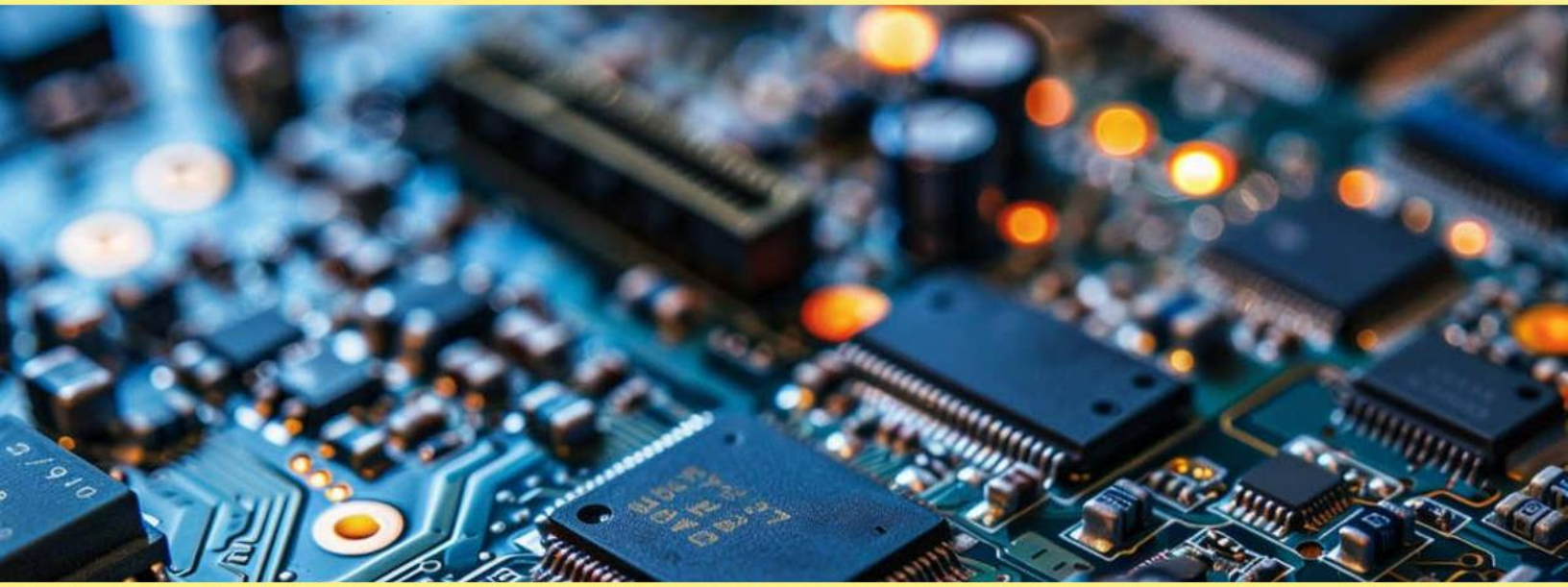
CPI A2 – PROJET SYSTÈMES EMBARQUÉS

LIVRABLE 4 – DOCUMENTATION



KHALDI Zakaria
KNIEFELD - LOINTIER Grégoire
GASPARIN Raphael
FILOTHEOU Nicolas
TURC Natan
RIVES Pierre-Elyan

CESI
ÉCOLE D'INGÉNIEURS

Sommaire

1. Introduction 2
1.1. Contexte 2
1.2. Objectifs 2
1.3. Problématique 2
1.4. Attendus du livrable 2
2. Documentation technique 3
2.1. Matériel et interfaces 3
2.2. Fonctionnement global et flux d'information 4
2.3. Architecture générale du programme 5
2.3.1. Constantes et variables globales 8
2.3.2. Signalisation par la LED 9
2.3.3. Lecture des boutons 9
2.3.4. Acquisition des mesures 10
2.3.5. Enregistrement sur la carte SD 11
2.3.6. Modes et machine à états 12
2.3.7. Initialisation et boucle principale 13
2.4. Performances du système 15
2.5. Choix et hypothèses 16
3. Documentation utilisateur : guide du superviseur 17
3.1. Présentation de la station 17
3.2. Mise en service 17
3.3. Lire la LED 18
3.4. Changer de mode 18
3.5. Utiliser chaque mode 19
3.6. Récupérer et lire les données 19
3.7. Messages de la console série 19
3.8. En cas d'erreur 20
3.9. Performances à connaître 20
4. Conclusion 21

1. Introduction

1.1. Contexte

L'Agence Internationale pour la Vigilance Météorologique (AIVM) veut mieux anticiper la formation des cyclones et des autres catastrophes naturelles en mer. Pour cela, elle prévoit d'équiper des navires de stations météo embarquées. Plusieurs compagnies de transport maritime ont accepté d'en installer sur leurs bateaux. À terme, ces navires pourront échanger leurs données pour améliorer les prévisions.

Notre startup est chargée de réaliser le prototype de cette station, baptisée *Worldwide Weather Watcher* (WWW). Ce prototype repose sur une carte STM32 Nucleo L476RG (microcontrôleur Cortex-M4), à laquelle sont reliés :

- Des capteurs de pression atmosphérique, de température de l'air et d'hygrométrie (module BME680), de luminosité et de position GPS ;
- Une horloge RTC pour dater les mesures ;
- Un lecteur de carte SD pour les sauvegarder ;
- Une LED RGB et deux boutons poussoirs pour échanger avec l'utilisateur.

1.2. Objectifs

Ce livrable poursuit deux objectifs. Le premier est de présenter les choix opérés, c'est-à-dire le fonctionnement global du système, le trajet des informations entre les composants et l'architecture générale du programme. Le second est d'informer l'utilisateur final, le superviseur de la station à bord, sur la manière de l'utiliser et sur les performances qu'il peut en attendre.

1.3. Problématique

Comment décrire le fonctionnement et l'architecture de la station météo de façon à ce qu'un technicien puisse la maintenir et la faire évoluer, et qu'un superviseur puisse l'utiliser au quotidien en connaissant ses limites ?

1.4. Attendus du livrable

L'énoncé demande deux documents :

- Une **documentation technique** : fonctionnement global du système avec un schéma des flux d'information, et architecture générale du programme ;
- Une **documentation utilisateur** : guide pour l'utilisateur final (superviseur).

Le document est donc organisé en deux grandes parties. La partie 2 constitue la documentation technique ; la partie 3 constitue le guide du superviseur et peut être imprimée seule.

2. Documentation technique

2.1. Matériel et interfaces

Le programme final est écrit avec le framework Arduino. Il s'appuie sur les bibliothèques *Adafruit_BME280*, *ChainableLED*, *Wire*, *RTClib*, *SPI*, *SD*, *SoftwareSerial* et *TinyGPS++*. Chaque composant est relié à une broche ou à un bus précis, défini par des constantes en tête du programme.

Composant	Rôle	Liaison	Pin ou adresse dans le programme
Capteur BME280	Température de l'air, pression, humidité	I2C	adresse 0x76 (ADR_BME280)
Capteur de luminosité	Luminosité ambiante (valeur 0 à 1023)	Analogique	A0 (PIN_LUMINOSITE)
GPS	Latitude et longitude	Série logicielle à 9600 bauds	RX 2, TX 3 (PIN_GPS_RX, PIN_GPS_TX)
Horloge RTC DS1307	Date et heure	I2C	adresse 0x68 (ADR_RTC)
Lecteur de carte SD	Enregistrement des mesures	SPI	sélection 4 (PIN_SD_CS)
LED RGB chaînable	Couleur du mode et codes d'erreur	2 fils (horloge + données)	8 et 9 (PIN_LED_CLK, PIN_LED_DATA)
Bouton vert	Passage en mode économique	Numérique, résistance de tirage interne	5 (PIN_BTN_VERT)
Bouton rouge	Configuration au démarrage, maintenance, retour	Numérique, résistance de tirage interne	6 (PIN_BTN_ROUGE)
Console série	Affichage des mesures et des messages	USB série à 9600 bauds	Serial

Tableau n°1 : Composants de la station et liaisons utilisées par le programme

La carte qui exécute ce programme est un Arduino Uno R3. Les boutons sont configurés en entrée avec résistance de tirage interne (INPUT_PULLUP) : un bouton appuyé est donc lu à l'état bas (BTN_APPUYE vaut LOW).

2.2. Fonctionnement global et flux d'information

Le flux d'information décrit le trajet d'une donnée depuis le composant qui la produit jusqu'à la sortie qui la restitue. La figure n°1 présente ce flux pour la station, en trois niveaux : les entrées en haut, le programme au centre et les sorties en bas.

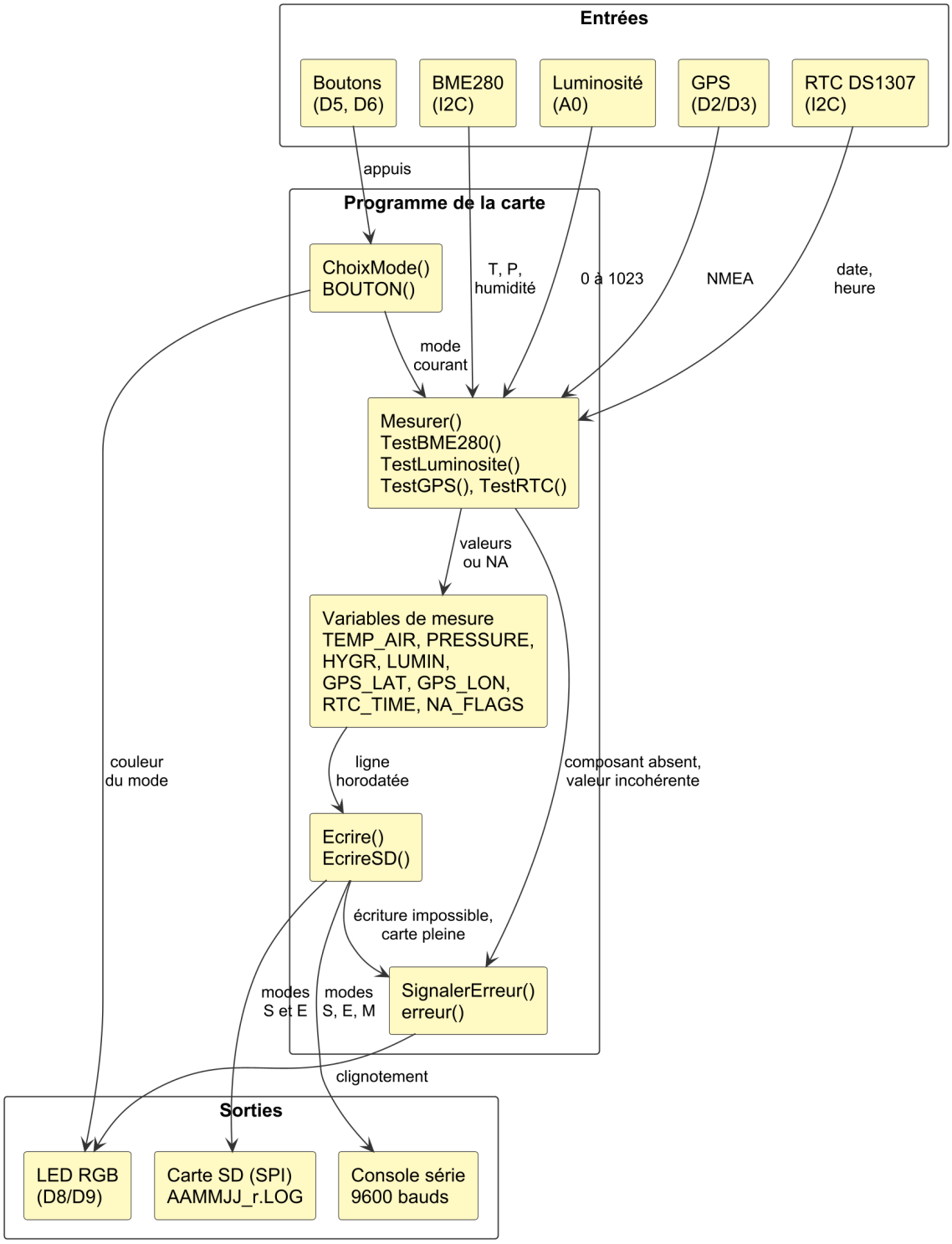


Figure n°1 : Flux d'information de la station météo

Les boutons sont lus par *ChoixMode()*, qui détermine le mode actuel (standard S, économique E, maintenance M ou configuration C) et donc ce que la station fait au relevé suivant.

À chaque relevé, *Mesurer()* interroge le BME280 pour la température, la pression et l'humidité, le capteur de luminosité, le GPS et l'horloge RTC. Les valeurs sont rangées dans des variables globales. Lorsqu'une valeur manque ou est incohérente, un bit est mis à 1 dans NA_FLAGS pour que la ligne écrite affiche « NA » à sa place.

Ecrire() transforme ensuite ces variables en une ligne de texte horodatée. Cette ligne est envoyée sur la carte SD par *EcrireSD()* en modes standard et économique, et sur la console série dans les modes qui prennent des mesures.

Enfin, la LED reçoit deux types d'information :

- la couleur fixe du mode, envoyée par la fonction du mode
- le clignotement d'erreur, envoyé par *erreur()* lorsqu'une anomalie a été signalée pendant le relevé par *SignalerErreur()*

2.3. Architecture générale du programme

Le programme est organisé en un seul fichier source, découpé en blocs de fonctions qui ont chacun un rôle. La figure n°2 présente l'arbre des appels depuis les deux points d'entrée Arduino, *setup()*, exécutée une fois à la mise sous tension, et *loop()*, exécutée en boucle ensuite.

Le programme se décompose en cinq blocs : la gestion des modes, l'acquisition, l'enregistrement, la signalisation et le diagnostic mémoire.

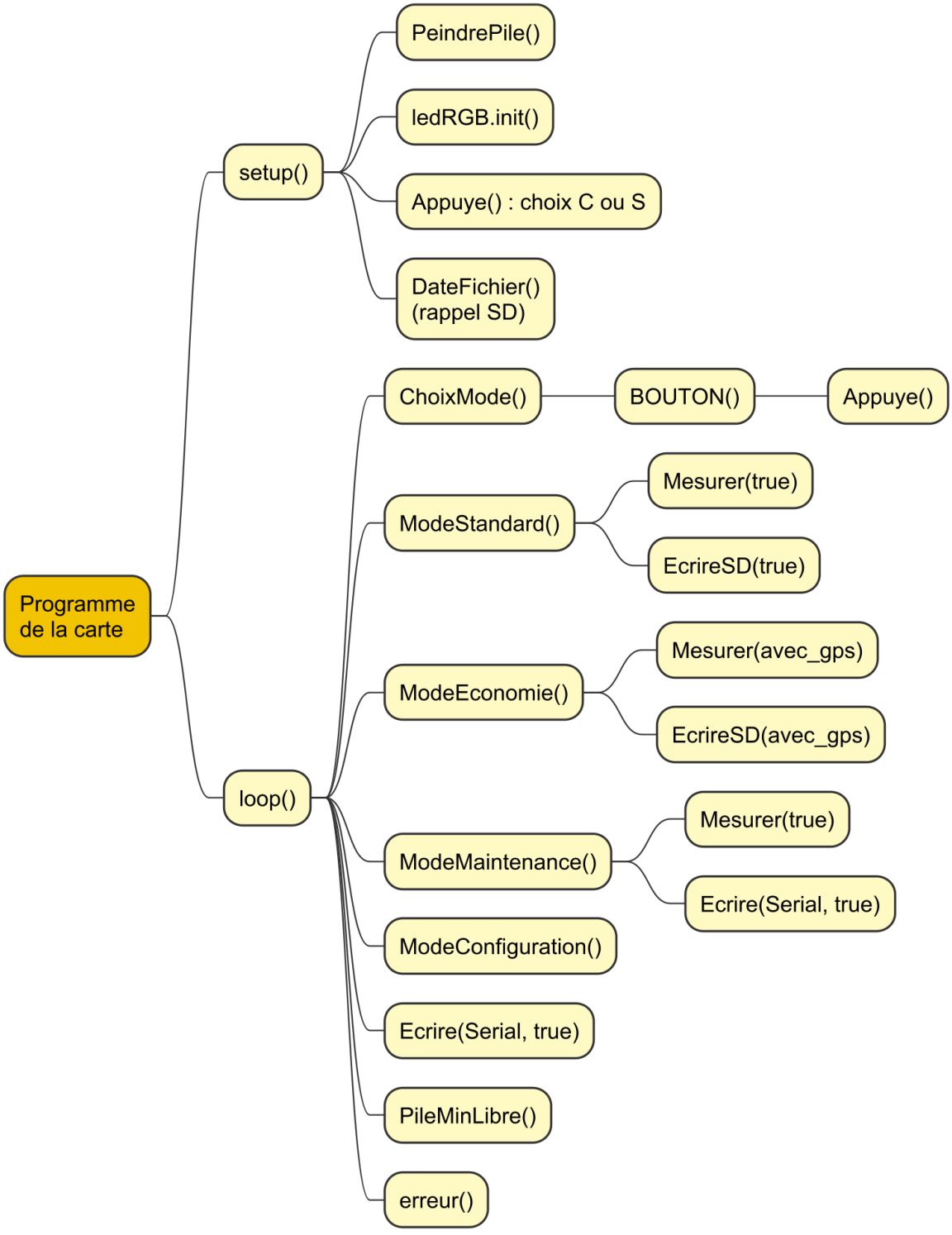


Figure n°2 : Arbre des appels depuis setup() et loop()

On y lit que *loop()* ne fait qu'enchaîner la lecture des boutons, la fonction du mode actuel, l'affichage sur la console et la gestion de l'erreur. Le travail est délégué à des fonctions de service, détaillées sur la figure n°3.

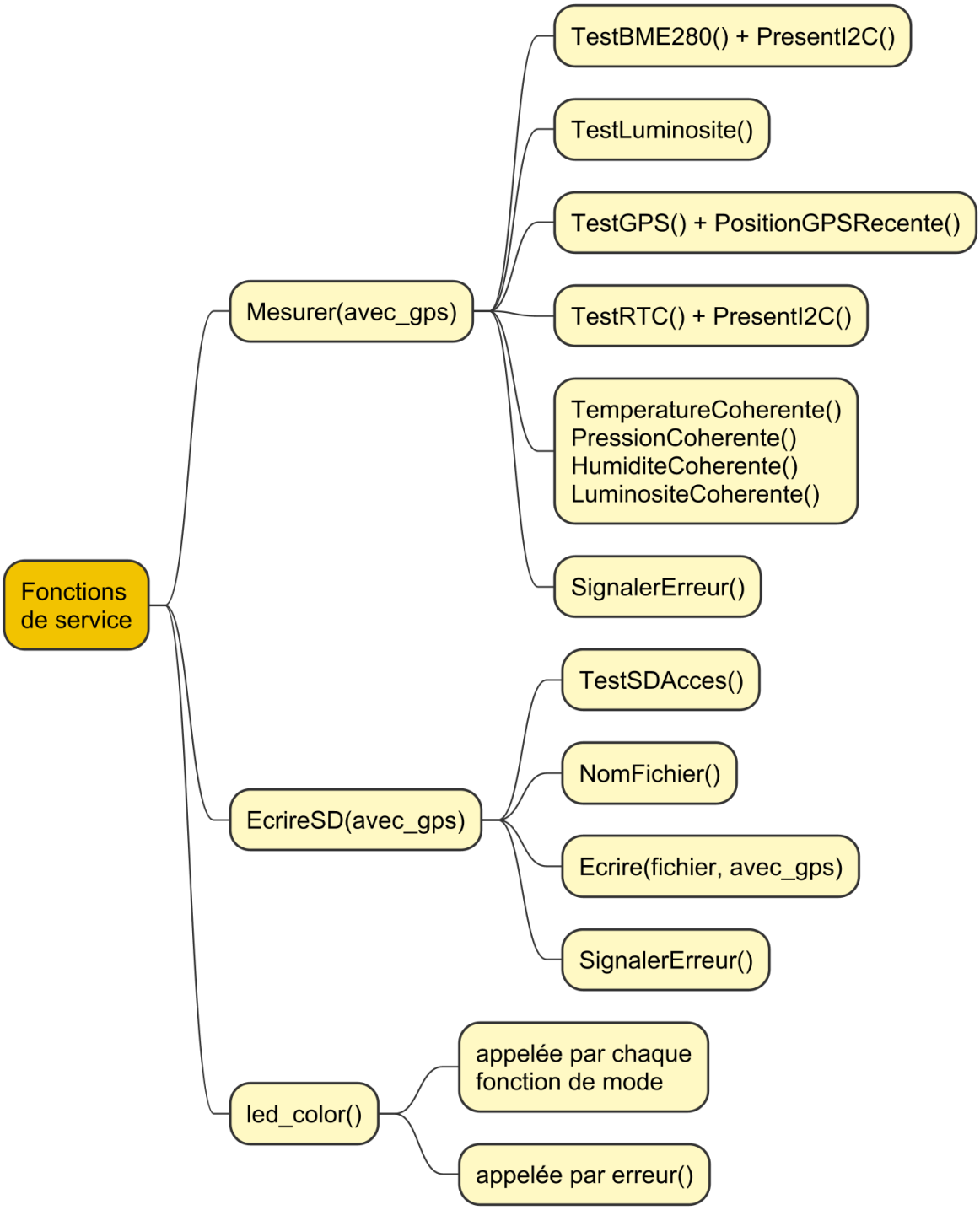


Figure n°3 : Fonctions de service appelées par les modes

2.3.1. Constantes et variables globales

Les paramètres de fonctionnement et les seuils de cohérence sont déclarés en tête du programme (Tableau n°2). Les seuils sont des constantes : ils sont fixés à la compilation.

Nom	Valeur dans le programme	Rôle
LOG_INTERVAL	600 à la déclaration (S), 1200 (E), 1800 (C)	Intervalle entre deux relevés, en secondes
FILE_MAX_SIZE	2048	Taille, en octets, à partir de laquelle on passe à la révision suivante du fichier
TIMEOUT	30	Délai maximal, en secondes, accordé à un composant pour répondre
MIN_TEMP_AIR / MAX_TEMP_AIR	−10 / 60	Plage de température cohérente (°C)
PRESSURE_MIN / PRESSURE_MAX	850 / 1080	Plage de pression cohérente (hPa)
LUMIN_LOW / LUMIN_HIGH	255 / 768	Seuils de luminosité déclarés, non utilisés dans les calculs
HYGR_MINT / HYGR_MAXT	0 / 50	Plage de température déclarée pour l'humidité, non utilisée dans les calculs
DUREE_INACTIVITE_CONFIG	30 min (en millisecondes)	Durée sans appui après laquelle le mode configuration repasse en standard

Tableau n°2 : Paramètres et seuils définis dans le programme

Les dernières mesures sont gardées dans TEMP_AIR, PRESSURE, HYGR, LUMIN, GPS_LAT, GPS_LON et RTC_TIME. L'état du système tient dans quelques variables : MODE et MODE_MEMOIRE (un caractère S, E, M ou C), num_revision (numéro du fichier en cours), COMPTEUR_ECO (alternance du GPS en mode économique), dernier_releve et premier_releve (déclenchement des relevés), ETAT_BOUTON (dernier appui lu) et erreur_courante (première erreur du relevé).

2.3.2. Signalisation par la LED

Les couleurs et les codes d'erreur sont décrits par deux structures, *Couleur* (trois composantes rouge, vert, bleu) et *Erreur* (deux couleurs et deux durées en millisecondes). Les six codes d'erreur sont déclarés une seule fois, sous forme de tableau de constantes :

```
typedef struct { const Couleur *c1; const Couleur *c2; uint16_t d1; uint16_t d2; } Erreur;

const Erreur ERR_RTC         PROGMEM = { &ROUGE, &BLEU,  1000, 1000 };
const Erreur ERR_GPS         PROGMEM = { &ROUGE, &JAUNE, 1000, 1000 };
const Erreur ERR_ACCES_CAP   PROGMEM = { &ROUGE, &VERT,  1000, 1000 };
const Erreur ERR_VALEURS_CAP PROGMEM = { &ROUGE, &VERT,  1000, 2000 };
const Erreur ERR_FULL_SD     PROGMEM = { &ROUGE, &BLANC, 1000, 1000 };
const Erreur ERR_WRITE_SD    PROGMEM = { &ROUGE, &BLANC, 1000, 2000 };

const Erreur *erreur_courante = NULL;

void SignalerErreur(const Erreur *e)
{
  if (erreur_courante == NULL) erreur_courante = e;
}

void erreur()
{
  led_color((const Couleur *)pgm_read_word(&erreur_courante->c1));
  delay(pgm_read_word(&erreur_courante->d1));
  led_color((const Couleur *)pgm_read_word(&erreur_courante->c2));
  delay(pgm_read_word(&erreur_courante->d2));
}
```

La fonction *SignalerErreur()* ne retient que la première erreur rencontrée pendant un relevé : une erreur suivante ne remplace pas la précédente. La fonction *erreur()* affiche la première couleur pendant d1, puis la seconde pendant d2. Ajouter un nouveau code d'erreur revient donc à ajouter une ligne à ce tableau, sans modifier *erreur()*.

2.3.3. Lecture des boutons

La fonction *BOUTON()* renvoie l'un des cinq états BTN_N (aucun appui), BTN_RC / BTN_VC (appui court rouge ou vert) et BTN_RL / BTN_VL (appui long rouge ou vert) :

```
uint8_t BOUTON()
{
  uint8_t pin;
  if (Appuye(PIN_BTN_ROUGE)) pin = PIN_BTN_ROUGE;
  else if (Appuye(PIN_BTN_VERT)) pin = PIN_BTN_VERT;
  else return BTN_N;

  delay(50);
  if (!Appuye(pin)) return BTN_N;

  chrono = millis();
  while (millis() - chrono < 5000UL)
  {
    if (!Appuye(pin))
    {
      delay(50);
      return (pin == PIN_BTN_ROUGE) ? BTN_RC : BTN_VC;
    }
  }
  while (Appuye(pin)) {}
  delay(50);
  return (pin == PIN_BTN_ROUGE) ? BTN_RL : BTN_VL;
}
```

Un appui est confirmé après 50 ms, pour ignorer les rebonds du contact. Le bouton est ensuite surveillé pendant 5 s : s'il est relâché avant, l'appui est court ; sinon, la fonction attend le relâchement et renvoie un appui long. Le changement de mode a donc lieu au relâchement du bouton, et non au bout des 5 s. Pendant cette attente, le programme ne fait rien d'autre.

2.3.4. Acquisition des mesures

La fonction *Mesurer(avec_gps)* remet NA_FLAGS à zéro, puis interroge les composants dans l'ordre BME280, luminosité, GPS (si avec_gps est vrai) et RTC. Chaque composant passe d'abord par une fonction de test qui répète la tentative jusqu'à TIMEOUT secondes :

- *TestBME280()* et *TestRTC()* vérifient que le composant répond sur le bus I2C (*PresentI2C()*), puis l'initialisent une seule fois. Si l'horloge n'est pas lancée, *TestRTC()* la règle sur la date et l'heure de compilation du programme ;
- *TestGPS()* décode les caractères reçus et renvoie vrai dès qu'une position récente (moins de 2 s) est valide. Au bout du délai, il renvoie vrai si au moins une trame correcte a été reçue, même sans position ;
- *TestLuminosite()* lit l'entrée analogique.

Si un composant ne répond pas, ses grandeurs sont marquées NA et une erreur d'accès est signalée (ERR_ACCES_CAP, ERR_GPS ou ERR_RTC). Si le BME280 répond mais qu'une valeur sort de sa plage de cohérence (Tableau n°2, et 0 à 100 % pour l'humidité), la grandeur est marquée NA et l'erreur ERR_VALEURS_CAP est signalée. Un GPS qui envoie des trames sans position valide donne NA, sans erreur.

Voici la fonction *TestLuminosite(), le fonctionnement est similaire pour les autres fonctions de tests :*

```
bool TestLuminosite()
{
  chrono = millis()

  while (millis() - chrono < TIMEOUT * 1000UL)
  {
     LUMIN = analogRead(PIN_LUMINOSITE);
     if (LUMIN <= 1023)
     {
        return true;
     }
  }
  return false;
}
```

2.3.5. Enregistrement sur la carte SD

La fonction *Ecrire(sortie, avec_gps)* écrit une ligne complète sur une sortie quelconque (fichier ou console série), ce qui évite de dupliquer le format. *EcrireSD()* gère le fichier :

```
void EcrireSD(bool avec_gps)
{
  if (!TestSDAcces())
  {
    SignalerErreur(&ERR_WRITE_SD);
    return;
  }
  NomFichier();
  File fichier = SD.open(FILE_PATH, FILE_WRITE);
  if (!fichier)
  {
    SignalerErreur(&ERR_WRITE_SD);
    return;
  }
  Ecrire(fichier, avec_gps);
  if (fichier.size() >= FILE_MAX_SIZE)
  {
    if (num_revision < 9)
    {
      num_revision += 1;
    }
    else SignalerErreur(&ERR_FULL_SD);
  }
  fichier.close();
}
```

NomFichier() construit le nom AAMMJJ_r.LOG à partir de la date de l'horloge et du numéro de révision r (0 à 9). Chaque ligne est ajoutée en fin de fichier, puis le fichier est fermé, ce qui limite le risque de fichier abîmé si la carte est retirée entre deux relevés. Lorsque le fichier atteint FILE_MAX_SIZE octets, les lignes suivantes vont dans la révision suivante ; après la révision 9, l'erreur « carte SD pleine » est signalée. Le numéro de révision n'est remis à zéro qu'au redémarrage de la carte, et non au changement de jour. La date et l'heure de création des fichiers sont fournies à la bibliothèque SD par *DateFichier()*.

2.3.6. Modes et machine à états

La figure n°4 présente les modes et les transitions programmés dans *ChoixMode()*.

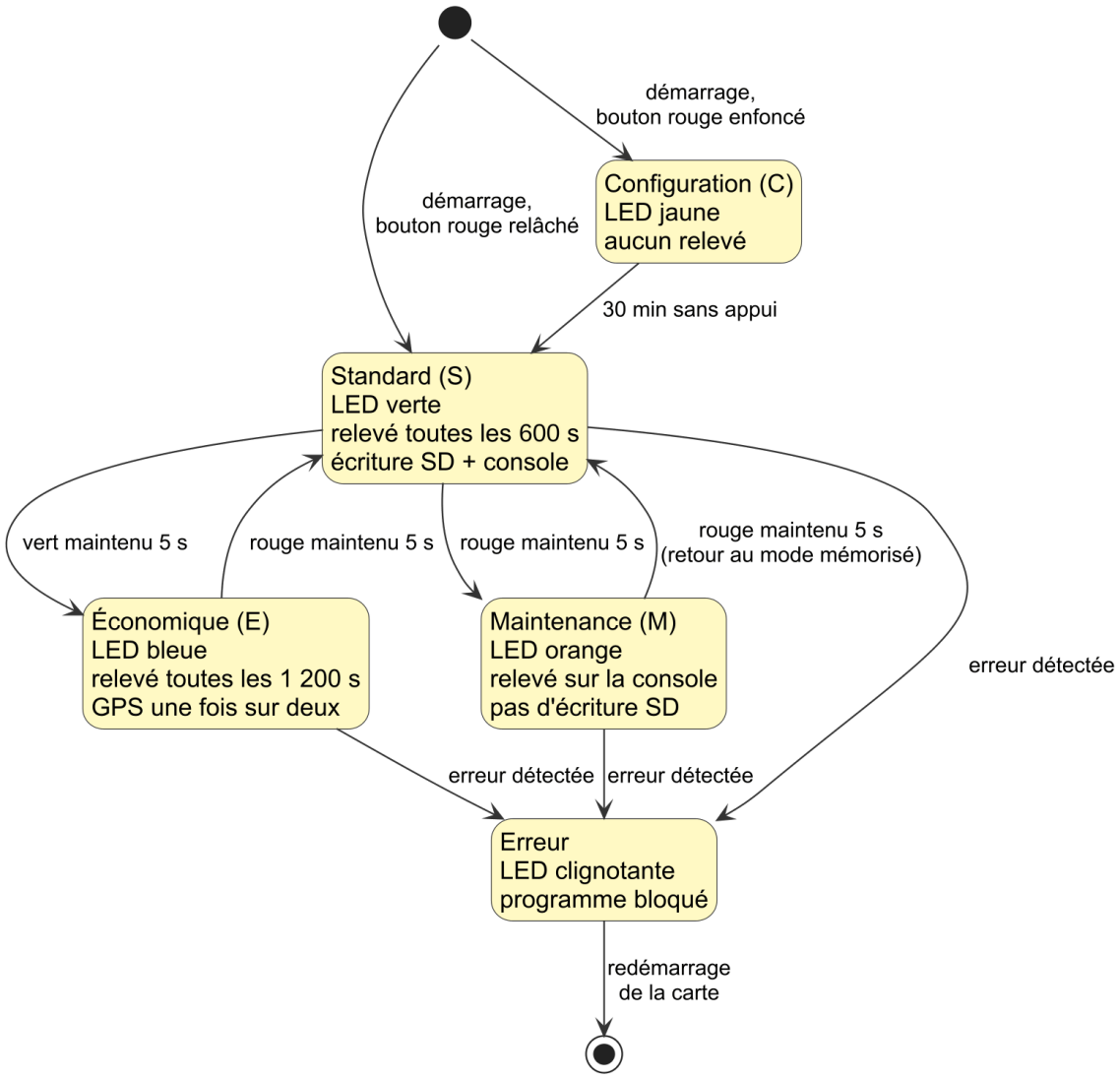


Figure n°4 : Machine à états des modes de fonctionnement programmés

Les appuis courts ne provoquent aucun changement de mode. Chaque fonction de mode allume d'abord la LED de sa couleur, puis :

- *ModeStandard()* fixe LOG_INTERVAL à 600 s, mesure avec le GPS et écrit sur la carte SD ;
- *ModeEconomie()* fixe LOG_INTERVAL à 1 200 s et ne lit le GPS qu'un relevé sur deux, grâce à COMPTEUR_ECO ;
- *ModeMaintenance()* mesure et affiche la ligne sur la console série, sans écrire sur la carte SD ;
- *ModeConfiguration()* fixe LOG_INTERVAL à 1 800 s et ne prend aucune mesure.

2.3.7. Initialisation et boucle principale

La fonction *setup()* remplit la zone libre de la pile avec un motif connu (*PeindrePile()*), initialise la LED, les deux liaisons série à 9600 bauds, le bus I2C et les boutons, choisit le mode de départ et initialise la carte SD. La fonction *loop()* est ensuite exécutée en boucle :

```
void loop()
{
  char ancien_mode = MODE;
  ChoixMode();
  if (ETAT_BOUTON != BTN_N)
  {
    Serial.print(F("Bouton : "));
    Serial.print(ETAT_BOUTON);
    Serial.print(F(" -> mode "));
    Serial.println(MODE);
  }
  if (premier_releve || MODE != ancien_mode || millis() - dernier_releve >= LOG_INTERVAL * 1000UL)
  {
    premier_releve = false;
    dernier_releve = millis();
    erreur_courante = NULL;
    if (MODE == 'S')
    {
      ModeStandard();
    }
    else if (MODE == 'E')
    {
      ModeEconomie();
    }
    else if (MODE == 'M')
    {
      ModeMaintenance();
    }
    else if (MODE == 'C')
    {
      ModeConfiguration();
    }
    if (MODE == 'S' || MODE == 'E')
    {
      Ecrire(Serial, true);
    }
    Serial.print(F("Le mode "));
    Serial.print(MODE);
    Serial.print(F(" est termine (pile : "));
    Serial.print(PileMinLibre());
    Serial.println(F(" octets libres au minimum)"));
  }
  if (erreur_courante != NULL)
  {
    while (true)
    {
      erreur();
    }
  }
  else
  {
    delay(100);
  }
}
```

À chaque tour, la boucle lit les boutons. Un relevé est lancé au premier tour, à chaque changement de mode et à chaque fin d'intervalle LOG_INTERVAL. Après le relevé, la boucle affiche sur la console la quantité minimale de pile restée libre depuis le démarrage, calculée par *PileMinLibre()* en cherchant la plus longue zone qui porte encore le motif 0xAA.

Si une erreur a été signalée, le programme entre dans une boucle sans fin qui fait clignoter la LED : la station ne prend plus de mesures tant qu'elle n'a pas été redémarrée.

La figure n°5 résume ce déroulement.

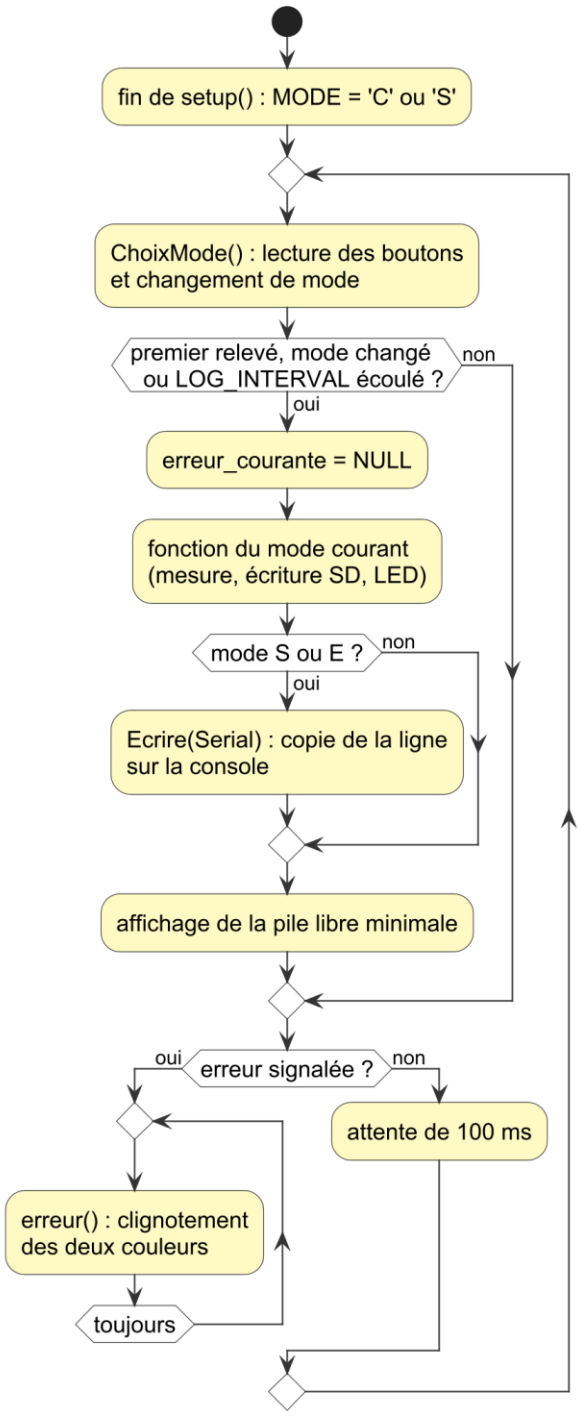


Figure n°5 : Déroulement de la boucle principale

2.4. Performances du système

Les performances qui découlent directement du programme sont rassemblées dans le Tableau n°3. Elles sont lues dans le code ; elles ne remplacent pas des mesures sur la maquette.

Caractéristique	Valeur d'après le programme
Intervalle entre deux relevés	600 s en mode standard, 1 200 s en mode économique
Lecture du GPS	à chaque relevé en mode standard et maintenance, un relevé sur deux en mode économique
Délai maximal d'attente d'un composant	30 s (TIMEOUT) par composant
Durée d'un relevé si tous les composants manquent	jusqu'à environ 30 s pour chacun du BME280, du GPS et de la RTC, plus 30 s pour la carte SD si elle n'est pas initialisée, soit environ 2 min
Appui long	au moins 5 s, pris en compte au relâchement
Retour automatique depuis la configuration	30 min sans appui
Taille d'un fichier	2 048 octets, dépassée au plus d'une ligne
Nombre de fichiers	10 révisions (_0 à _9) depuis le démarrage, soit environ 20 ko ; le numéro n'est pas remis à zéro au changement de jour
Clignotement d'erreur	1 s / 1 s, ou 1 s / 2 s pour les données incohérentes et l'écriture SD

Tableau n°3 : Performances déduites du programme

Le programme affiche après chaque relevé la pile libre minimale, ce qui permet de vérifier que la mémoire vive ne s'épuise pas. Les valeurs mesurées sur la maquette sont reportées dans le Tableau n°4.

Grandeur mesurée	Valeur
Pile libre minimale affichée sur la console	Avec 1457 o de RAM statique, il reste au plus 591 o
Mémoire programme et mémoire vive utilisées (compilation)	Flash : 29 874 / 32 256 o (92,6 %). RAM : 1 457 / 2 048 o (71,1 %).
Durée d'un relevé complet, composants présents	0,5 à 1,5 secondes
Longueur d'une ligne et nombre de lignes par fichier	Environ 157 o par ligne avec le GPS. Environ 119 o sans le GPS. Soit 13 à 16 lignes par fichier.
Consommation et autonomie	Autonomie : environ 4–5 h avec une pile 9 V de 500 mAh, environ 17–20 h avec une batterie externe de 2000 mAh.

Tableau n°4 : Performances mesurées sur la maquette

2.5. Choix et hypothèses

Organisation du code. Nous avons regroupé le programme en blocs de fonctions courtes, chacune chargée d'une seule tâche (tester un composant, écrire une ligne, choisir un mode). Les quatre fonctions de mode réutilisent ainsi les mêmes fonctions *Mesurer()*, *Ecrire()* et *EcrireSD()*, avec un simple paramètre avec_gps pour le mode économique.

Économie de mémoire vive. Les couleurs et les codes d'erreur sont rangés en mémoire programme (PROGMEM) et les textes de la console sont entourés de F(), pour ne pas occuper la mémoire vive. Les valeurs absentes sont mémorisées dans un seul octet, NA_FLAGS, à raison d'un bit par grandeur. La mesure de la pile libre (*PeindrePile()*, *PileMinLibre()*) a été ajoutée pour contrôler cette ressource.

Une seule erreur affichée. Seule la première erreur d'un relevé est affichée, ce qui garde un code de LED lisible. L'erreur bloque ensuite la station jusqu'au redémarrage.

3. Documentation utilisateur : guide du superviseur

Ce guide s'adresse au superviseur chargé de la station à bord. Il décrit la mise en service, la lecture de la LED, l'usage des deux boutons, la récupération des données et la conduite à tenir en cas d'erreur.

3.1. Présentation de la station

La station mesure la température de l'air, la pression atmosphérique, l'humidité, la luminosité et la position du navire, puis enregistre ces mesures avec la date et l'heure sur une carte SD. Le superviseur dispose de trois éléments :

- Un **bouton rouge** et un **bouton vert**, pour changer de mode ;
- Une **LED de couleur**, qui indique le mode en cours ou une erreur ;
- Une **console série**, sur un ordinateur relié par USB, pour lire les mesures en direct.

3.2. Mise en service

1. Vérifier que les capteurs, le GPS, l'horloge, la LED et le lecteur de carte SD sont branchés sur les broches du Tableau n°1.
2. Insérer une carte SD dans le lecteur.
3. Mettre la carte sous tension sans appuyer sur les boutons : la LED s'allume en vert et un premier relevé est pris tout de suite. La station est en mode standard.
4. Pour lire les mesures en direct, ouvrir un moniteur série à **9600 bauds**.

3.3. Lire la LED

En fonctionnement normal, la LED reste allumée d'une couleur fixe qui indique le mode (Tableau n°5).

Couleur fixe	Mode	Ce que fait la station
Vert	Standard	Mesure toutes les 10 minutes et enregistre sur la carte SD
Bleu	Économique	Mesure toutes les 20 minutes, GPS une fois sur deux, enregistre sur la carte SD
Orange	Maintenance	Mesure et affiche sur la console, n'écrit plus sur la carte SD
Jaune	Configuration	Ne mesure pas ; repasse seule en standard après 30 minutes sans appui

Tableau n°5 : Couleurs de la LED en fonctionnement normal

En cas d'erreur, la LED clignote en alternant le rouge et une seconde couleur qui indique l'origine du problème (Tableau n°6).

Clignotement	Erreur
Rouge 1 s / bleu 1 s	Horloge RTC inaccessible
Rouge 1 s / jaune 1 s	GPS inaccessible (aucune trame reçue)
Rouge 1 s / vert 1 s	Capteur inaccessible (BME280 ou luminosité)
Rouge 1 s / vert 2 s	Valeur de capteur incohérente
Rouge 1 s / blanc 1 s	Carte SD pleine
Rouge 1 s / blanc 2 s	Carte SD absente ou écriture impossible

Tableau n°6 : Codes d'erreur de la LED

3.4. Changer de mode

Un **appui long** consiste à maintenir un bouton enfoncé au moins 5 secondes, puis à le relâcher : le changement de mode a lieu au relâchement. Un appui court n'a aucun effet. Le Tableau n°7 résume les actions possibles, que la figure n°4 représente aussi.

Mode actuel	Action	Nouveau mode
Station éteinte	Mise sous tension, bouton rouge enfoncé	Configuration (jaune)
Station éteinte	Mise sous tension, sans appui	Standard (vert)
Standard (vert)	Appui long sur le bouton vert	Économique (bleu)
Standard (vert)	Appui long sur le bouton rouge	Maintenance (orange)
Économique (bleu)	Appui long sur le bouton rouge	Standard (vert)
Maintenance (orange)	Appui long sur le bouton rouge	Retour au mode précédent (standard)
Configuration (jaune)	30 minutes sans appui	Standard (vert)

Tableau n°7 : Actions sur les boutons et changements de mode

À chaque changement de mode, la station prend immédiatement un relevé. Pendant un relevé, qui peut durer jusqu'à environ 2 minutes si des composants ne répondent pas, les boutons ne sont pas lus : il faut attendre la fin du relevé avant d'agir.

3.5. Utiliser chaque mode

Mode standard. C'est le mode d'usage par défaut. La station mesure toutes les 10 minutes et ajoute une ligne au fichier du jour sur la carte SD ; la même ligne est affichée sur la console.

Mode économique. Il réduit le nombre de relevés pour économiser l'énergie : un relevé toutes les 20 minutes, avec la position GPS une fois sur deux. Les lignes sans GPS ne contiennent pas les coordonnées.

Mode maintenance. Il sert à contrôler les capteurs et à retirer la carte SD sans risque. Les mesures s'affichent sur la console et plus rien n'est écrit sur la carte : on peut la retirer, puis la remettre avant de quitter le mode par un appui long rouge.

Mode configuration. Dans cette version, il n'accepte pas encore de commandes de réglage : la LED est jaune et aucune mesure n'est prise. Pour en sortir sans attendre 30 minutes, il suffit de redémarrer la station sans appuyer sur le bouton rouge.

3.6. Récupérer et lire les données

Les fichiers sont nommés AAMMJJ_r.LOG, où AAMMJJ est la date (année, mois, jour sur deux chiffres) et r le numéro de révision, de 0 à 9. Par exemple, les mesures du 5 octobre 2026 commencent dans 261005_0.LOG. Lorsqu'un fichier dépasse 2 048 octets, la station passe à la révision suivante (_1, _2, …). Le numéro de révision repart de 0 à chaque redémarrage, et il est conservé d'un jour à l'autre tant que la station n'est pas redémarrée. Chaque ligne a la forme suivante :

```
Date/Heure: <année>/<mois>/<jour> - <heure>:<minute>:<seconde> | GPS lat, long : <latitude>, <longitude> | Température : <°C> | Pression : <hPa> | Humidité : <%> | Luminosité : <0 à 1023>
```

Une valeur qui n'a pas pu être mesurée est remplacée par NA. La luminosité est une valeur brute : plus elle est élevée, plus la lumière est forte. Les jours, mois, heures, minutes et secondes sont écrits sans zéro devant (par exemple 9:3:7).

Pour récupérer les données, passer en mode maintenance (LED orange), retirer la carte SD, copier les fichiers .LOG sur un ordinateur, remettre la carte, puis revenir au mode précédent par un appui long rouge.

3.7. Messages de la console série

La console, réglée à 9600 bauds, affiche :

- les étapes du relevé (Mesurer : BME280..., Mesurer : luminosite..., Mesurer : GPS..., Mesurer : RTC..., Mesurer : fin) ;
- la ligne de mesures, au même format que sur la carte SD ;
- chaque appui pris en compte, sous la forme Bouton : <code> -> mode <lettre> (codes 1 et 3 : appui court rouge et vert ; 2 et 4 : appui long rouge et vert) ;
- après chaque relevé, Le mode <lettre> est termine (pile : <n> octets libres au minimum).

3.8. En cas d'erreur

Lorsqu'une erreur est affichée, la station s'arrête de mesurer et la LED clignote tant qu'elle n'a pas été redémarrée. Le Tableau n°8 donne la conduite à tenir.

Clignotement	Vérification à faire	Ensuite
Rouge / bleu	Branchement de l'horloge RTC sur le bus I2C	Redémarrer la station
Rouge / jaune	Branchement du GPS (le GPS peut mettre un certain temps pour s'initialiser : 2 à 3 minutes)	Redémarrer la station
Rouge / vert (temps égaux)	Branchement du capteur BME280 ou du capteur de luminosité	Redémarrer la station
Rouge / vert (vert plus long)	Conditions de mesure : température entre −10 et 60 °C, pression entre 850 et 1 080 hPa	Redémarrer la station ; si l'erreur revient, signaler le capteur
Rouge / blanc (temps égaux)	Carte SD pleine : récupérer les fichiers puis libérer de la place	Redémarrer la station
Rouge / blanc (blanc plus long)	Présence et insertion de la carte SD	Redémarrer la station

Tableau n°8 : Conduite à tenir en cas d'erreur

Un GPS qui n'a pas encore capté de satellites n'est pas une erreur : la station écrit NA, NA à la place des coordonnées et continue de fonctionner.

3.9. Performances à connaître

- Une mesure toutes les **10 minutes** en mode standard, toutes les **20 minutes** en mode économique.
- Un composant absent fait attendre la station jusqu'à **30 secondes** avant qu'elle ne le déclare en erreur.
- Un appui long dure **au moins 5 secondes**.
- La station peut écrire jusqu'à **10 fichiers de 2 Ko** (révisions 0 à 9) entre deux redémarrages, soit environ 20 Ko ; au-delà, elle signale « carte SD pleine ».

4. Conclusion

Ce livrable a permis de documenter la station Worldwide Weather Watcher telle qu'elle est programmée. Sur le plan technique, nous avons décrit le matériel et ses liaisons, le flux d'information depuis les capteurs jusqu'à la carte SD, la console et la LED, puis l'architecture du programme en cinq blocs de fonctions et la machine à états des modes. Les choix opérés, comme le regroupement des codes d'erreur dans un tableau de constantes, l'usage de la mémoire programme pour économiser la mémoire vive ou la fonction d'écriture commune au fichier et à la console, ont été justifiés.

Sur le plan de l'utilisation, le guide du superviseur explique comment démarrer la station, lire la LED, changer de mode avec deux boutons, récupérer les fichiers de mesures et réagir à chaque code d'erreur. Les performances qui découlent du programme (intervalles de 10 et 20 minutes, délai de 30 secondes par composant, capacité de 10 fichiers de 2 Ko entre deux redémarrages) y sont rappelées.